In [ ]:
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# 设置随机种子以保证可重复性
torch.manual_seed(42)
np.random.seed(42)
# 绘图设置
plt.rcParams['font.sans-serif'] = ['Noto Sans CJK JP']  # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False    # 用来正常显示负号
# 读取数据
df = pd.read_csv('Concrete_Data_Yeh.csv')
print("数据集形状:", df.shape)
df.head()
```

```
数据集形状: (1030, 9)
```

Out[ ]:

	cement	slag	flyash	water	superplasticizer	coarseaggregate	fineaggregate
0	540.0	0.0	0.0	162.0	2.5	1040.0	676.0
1	540.0	0.0	0.0	162.0	2.5	1055.0	676.0
2	332.5	142.5	0.0	228.0	0.0	932.0	594.0
3	332.5	142.5	0.0	228.0	0.0	932.0	594.0
4	198.6	132.4	0.0	192.0	0.0	978.4	825.5

导入所需库，由于采用的是linux系统需要进行绘图设置，然后在同一个目录下读取数据Concrete_Data_Yeh.csv，并打印数据集形状 然后读取前5行数据查看是否出错。

In [2]:
```python
# 数据基本信息
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1030 entries, 0 to 1029
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   cement            1030 non-null   float64
 1   slag              1030 non-null   float64
 2   flyash            1030 non-null   float64
 3   water             1030 non-null   float64
 4   superplasticizer  1030 non-null   float64
 5   coarseaggregate   1030 non-null   float64
 6   fineaggregate     1030 non-null   float64
 7   age               1030 non-null   int64
 8   csMPa             1030 non-null   float64
dtypes: float64(8), int64(1)
memory usage: 72.5 KB
```

查看数据基本信息

In [3]:
```python
# 描述性统计
df.describe()
```

Out[3]:

	cement	slag	flyash	water	superplasticizer	coars
count	1030.000000	1030.000000	1030.000000	1030.000000	1030.000000	1
mean	281.167864	73.895825	54.188350	181.567282	6.204660	
std	104.506364	86.279342	63.997004	21.354219	5.973841	
min	102.000000	0.000000	0.000000	121.800000	0.000000	
25%	192.375000	0.000000	0.000000	164.900000	0.000000	
50%	272.900000	22.000000	0.000000	185.000000	6.400000	
75%	350.000000	142.950000	118.300000	192.000000	10.200000	1
max	540.000000	359.400000	200.100000	247.000000	32.200000	1

查看数据数量，均值，最大最小值等统计信息

In [4]:
```python
# 检查缺失值
df.isnull().sum()
```

Out[4]:
```
cement              0
slag                0
flyash              0
water               0
superplasticizer    0
coarseaggregate     0
fineaggregate       0
age                 0
csMPa               0
dtype: int64
```

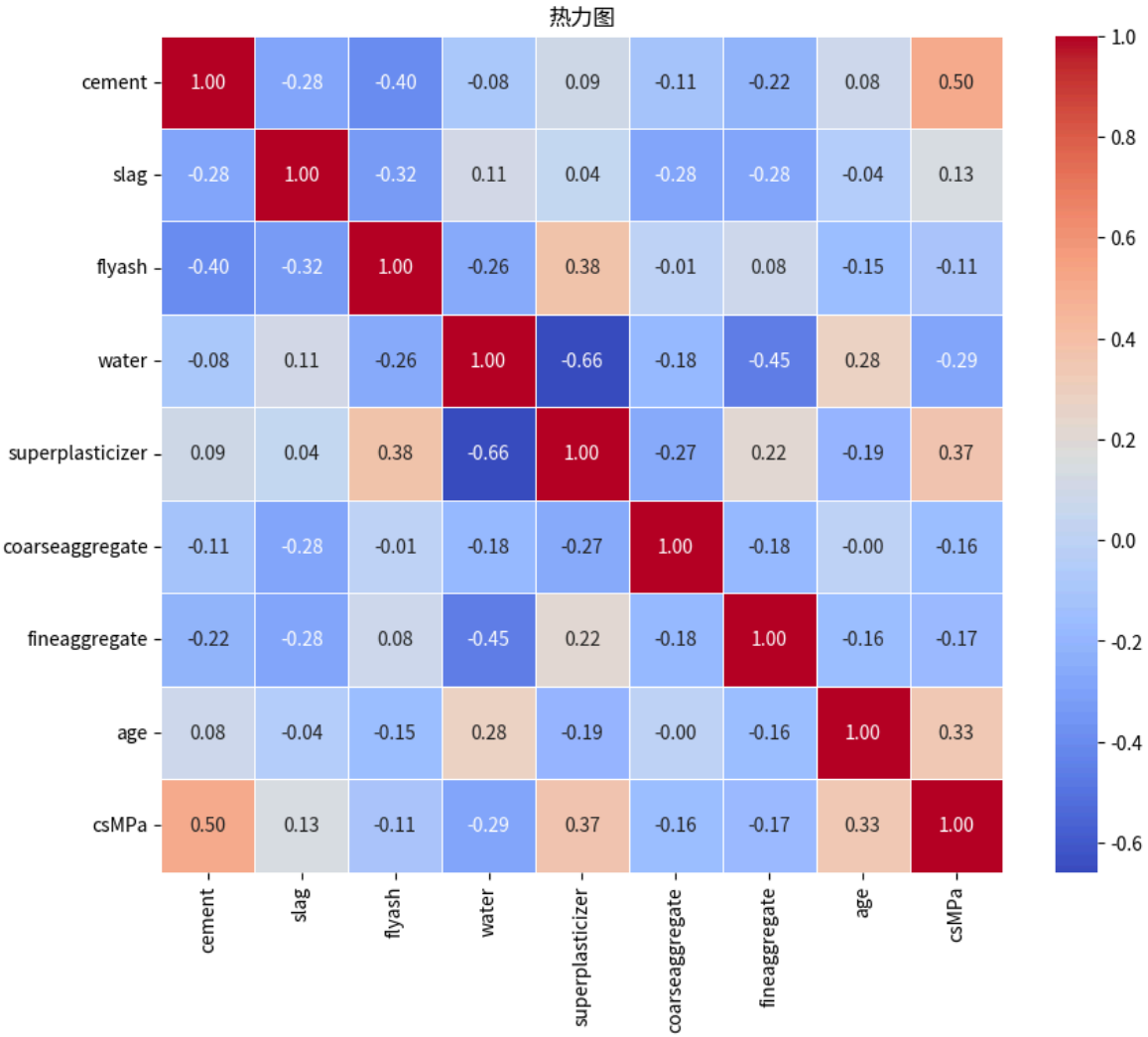
检查是否有缺失值，方便数据预处理

In [5]:
```python
# 计算所有特征与目标变量的相关系数
corr_with_target = df.corr()['csMPa'].sort_values(ascending=False)
print("各特征与抗压强度的相关系数：\n", corr_with_target)
```

```
各特征与抗压强度的相关系数：
 csMPa               1.000000
cement              0.497832
superplasticizer    0.366079
age                 0.328873
slag                0.134829
flyash             -0.105755
coarseaggregate    -0.164935
fineaggregate      -0.167241
water              -0.289633
Name: csMPa, dtype: float64
```

计算各特征与抗压强度的相关系数，可以看到cement，superplasticizer，age ，slag呈正相关。flyash，coarseaggregate，coarseaggregate，water 呈负相关。

In [6]:
```python
# 绘制相关性热力图
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title('热力图')
plt.show()
```



绘制热力图，直观得查看相关性。

In [7]:
```
feature_columns = ['cement', 'superplasticizer', 'age']
X = df[feature_columns].values
y = df['csMPa'].values.reshape(-1, 1)

# 划分训练集和测试集（前80%训练，后20%测试—按顺序划分）
split_idx = int(0.8 * len(X))
X_train, X_test = X[:split_idx], X[split_idx:]
y_train, y_test = y[:split_idx], y[split_idx:]

print(f"训练集样本数: {X_train.shape[0]}, 测试集样本数: {X_test.shape[0]}")
```

```
训练集样本数: 824, 测试集样本数: 206
```

分离特征和标签，采用了3个输入变量（'cement', 'superplasticizer', 'age'）来建模。然后划分训练集和测试集分别占80%和20%。 打印训练集和测试集样本看是否划分对。

In [8]:
```
# 数据标准化（Z-score）
scaler_X = StandardScaler()
scaler_y = StandardScaler()

X_train_scaled = scaler_X.fit_transform(X_train)
X_test_scaled = scaler_X.transform(X_test)

y_train_scaled = scaler_y.fit_transform(y_train)
y_test_scaled = scaler_y.transform(y_test)

# 转换为 PyTorch 张量
X_train_tensor = torch.tensor(X_train_scaled, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train_scaled, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test_scaled, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test_scaled, dtype=torch.float32)
```

数据预处理，先标准化再转换为pytorch张量方便后续处理。

In [9]:
```
# 定义网络结构
class ConcreteNN(nn.Module):
    def __init__(self, input_dim):
        super(ConcreteNN, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 1)
        )

    def forward(self, x):
        return self.net(x)

# 初始化模型
input_dim = X_train.shape[1]
model = ConcreteNN(input_dim)
print(model)
```

```
ConcreteNN(
  (net): Sequential(
    (0): Linear(in_features=3, out_features=64, bias=True)
    (1): ReLU()
    (2): Linear(in_features=64, out_features=32, bias=True)
    (3): ReLU()
    (4): Linear(in_features=32, out_features=1, bias=True)
  )
)
```

构建神经网络，采用两个隐藏层结构，激活函数使用ReLU激活函数，输入为8×64矩阵。然后初始化模型，并打印神经网络的结构

In [10]:
```
# 创建 DataLoader 以便批量训练
batch_size = 32
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
```

创建数据加载器以便批量训练

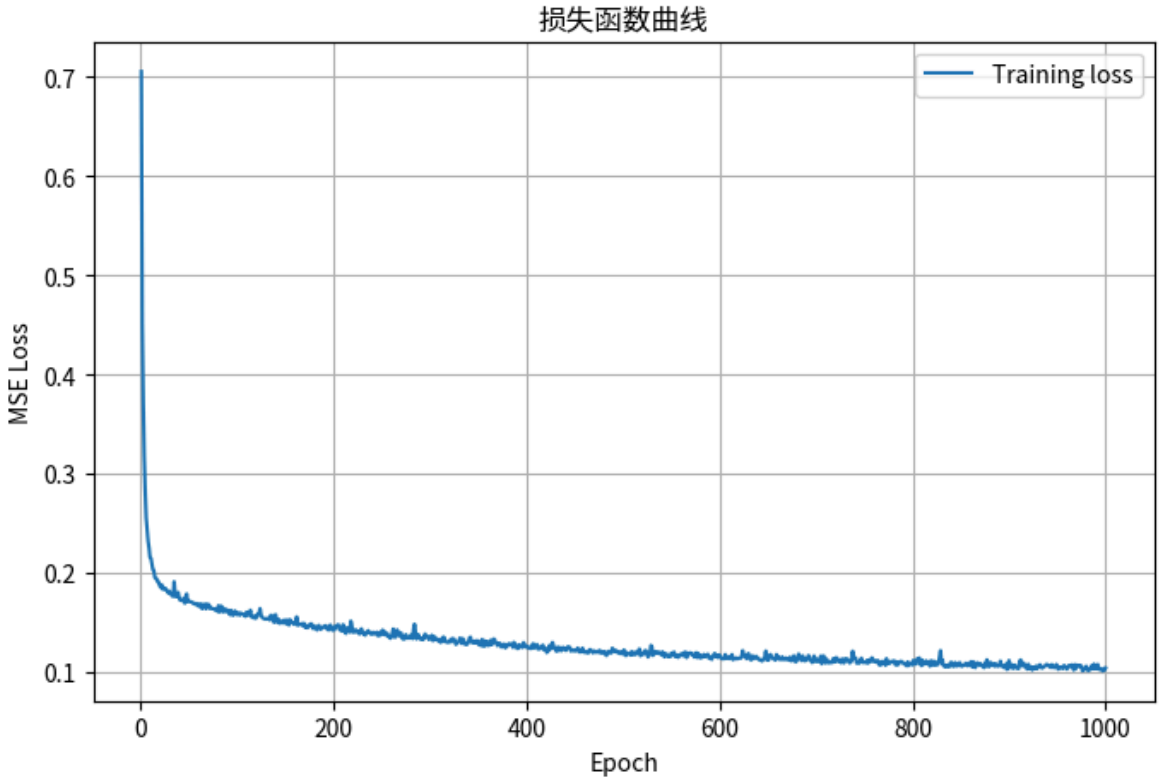
In [11]:
```
model = ConcreteNN(input_dim)
# 定义损失函数和优化器
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
num_epochs = 1000
train_losses = []

for epoch in range(num_epochs):
    model.train()
    epoch_loss = 0.0
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item() * X_batch.size(0)

    epoch_loss /= len(train_loader.dataset)
    train_losses.append(epoch_loss)

    if (epoch+1) % 20 == 0:
        print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {epoch_loss:.4f}')
plt.figure(figsize=(8,5))
plt.plot(range(1, num_epochs+1), train_losses, label='Training loss')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('损失函数曲线')
plt.legend()
plt.grid(True)
plt.show()
```

```
Epoch [20/1000], Loss: 0.1879
Epoch [40/1000], Loss: 0.1766
Epoch [60/1000], Loss: 0.1672
Epoch [80/1000], Loss: 0.1619
Epoch [100/1000], Loss: 0.1585
Epoch [120/1000], Loss: 0.1552
Epoch [140/1000], Loss: 0.1585
Epoch [160/1000], Loss: 0.1492
Epoch [180/1000], Loss: 0.1476
Epoch [200/1000], Loss: 0.1447
Epoch [220/1000], Loss: 0.1409
Epoch [240/1000], Loss: 0.1389
Epoch [260/1000], Loss: 0.1358
Epoch [280/1000], Loss: 0.1378
Epoch [300/1000], Loss: 0.1349
Epoch [320/1000], Loss: 0.1311
Epoch [340/1000], Loss: 0.1311
Epoch [360/1000], Loss: 0.1312
Epoch [380/1000], Loss: 0.1278
Epoch [400/1000], Loss: 0.1270
Epoch [420/1000], Loss: 0.1247
Epoch [440/1000], Loss: 0.1244
Epoch [460/1000], Loss: 0.1211
Epoch [480/1000], Loss: 0.1200
Epoch [500/1000], Loss: 0.1188
Epoch [520/1000], Loss: 0.1191
Epoch [540/1000], Loss: 0.1181
Epoch [560/1000], Loss: 0.1189
Epoch [580/1000], Loss: 0.1150
Epoch [600/1000], Loss: 0.1180
Epoch [620/1000], Loss: 0.1146
Epoch [640/1000], Loss: 0.1140
Epoch [660/1000], Loss: 0.1154
Epoch [680/1000], Loss: 0.1124
Epoch [700/1000], Loss: 0.1112
Epoch [720/1000], Loss: 0.1110
Epoch [740/1000], Loss: 0.1148
Epoch [760/1000], Loss: 0.1113
Epoch [780/1000], Loss: 0.1112
Epoch [800/1000], Loss: 0.1079
Epoch [820/1000], Loss: 0.1080
Epoch [840/1000], Loss: 0.1107
Epoch [860/1000], Loss: 0.1079
Epoch [880/1000], Loss: 0.1070
Epoch [900/1000], Loss: 0.1124
Epoch [920/1000], Loss: 0.1086
Epoch [940/1000], Loss: 0.1076
Epoch [960/1000], Loss: 0.1037
Epoch [980/1000], Loss: 0.1066
Epoch [1000/1000], Loss: 0.1048
```

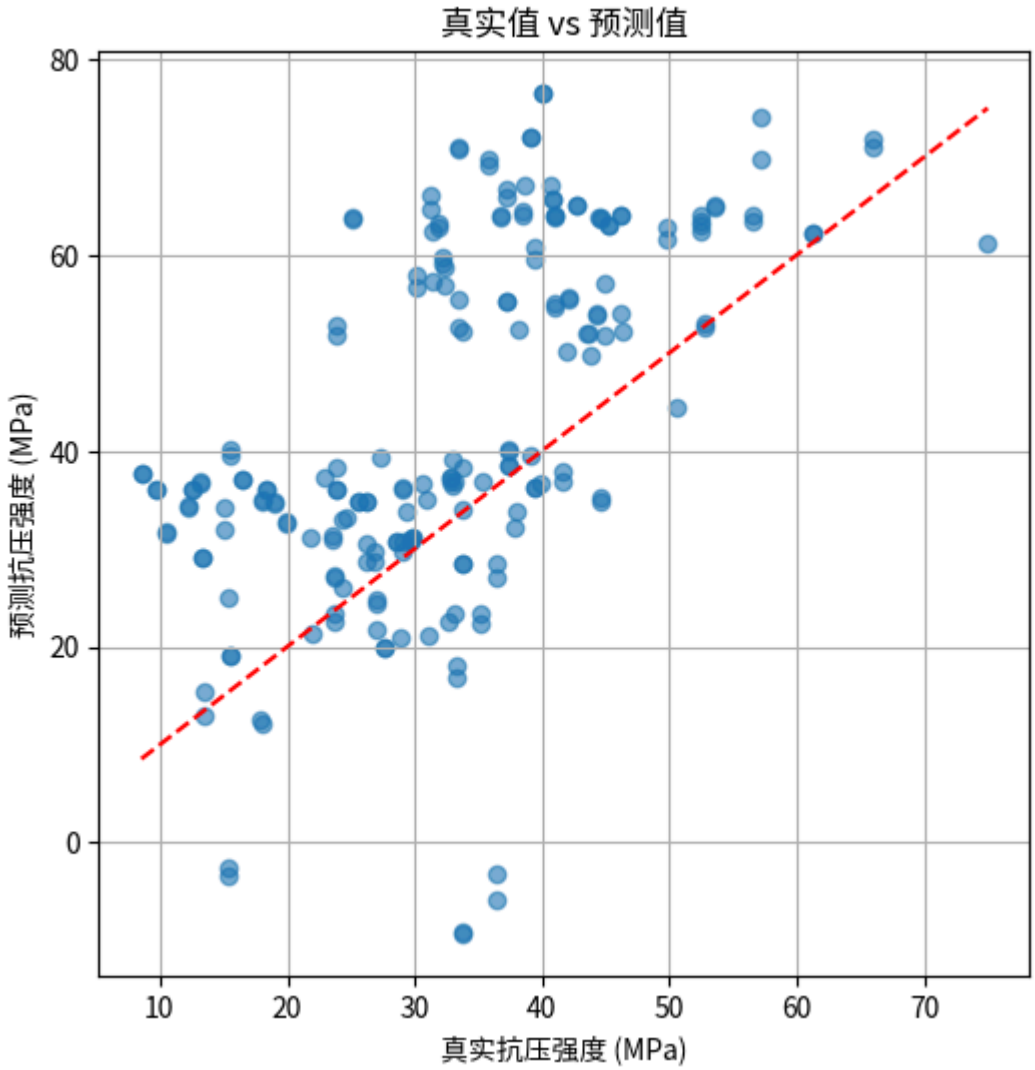


开始训练，使用均方损失函数，优化器采用Adam优化，参数设置：epoch：1000， 学习率：0.001。 采用反向传播算法进行梯度更新参数，然后绘制损失函数曲线

In [ ]:
```
# 测试集预测
model.eval()
with torch.no_grad():
    y_pred_scaled = model(X_test_tensor).numpy()
    y_pred = scaler_y.inverse_transform(y_pred_scaled)
    y_actual = scaler_y.inverse_transform(y_test_tensor.numpy())

# 计算均方误差
mse = mean_squared_error(y_actual, y_pred)
print(f"测试集均方误差 (MSE): {mse:.4f}")
# 绘制真实值与预测值散点图
plt.figure(figsize=(6,6))
plt.scatter(y_actual, y_pred, alpha=0.6)
plt.plot([y_actual.min(), y_actual.max()], [y_actual.min(), y_actual.max()], 'r-
plt.xlabel('真实抗压强度 (MPa)')
plt.ylabel('预测抗压强度 (MPa)')
plt.title('真实值 vs 预测值')
plt.grid(True)
plt.show()
```

```
测试集均方误差 (MSE): 326.0814
```



计算均方误差，可以看出测试集均方误差为326.0814. 然后根据数据，绘制真实值与预测值的散点图，散点越靠近红色虚线说明模型性能越好。 可以看出大部分散点都分布在红色虚线上方，说明模型性能一般。 神经网络是一种数据驱动模型，当输入特征较少时，模型容量（参数数量）可能无法充分利用数据中的复杂模式。 虽然保持网络结构相同（隐藏层64→32），但输入从8维降为3维，网络前端权重矩阵的尺寸大幅减小（8×64 vs 3×64）， 这限制了模型从数据中提取复杂特征的能力。增加特征维度为网络提供了更多原始信息，使其能够学习更丰富的表示。 在神经网络模型中，简单地依据单变量线性相关性筛选特征可能导致信息丢失，降低模型性能。 全部特征能提供更全面的信息，有利于神经网络学习复杂的非线性关系。